

一个不存在的专业。即被参照关系“专业”中一定存在一个元组，它的主码值等于该参照关系“学生”中的外码值。

对于例 2.2，按照参照完整性约束，“学号”和“课程号”属性也可以取两类值：空值或目标关系中已经存在的值。但由于这两个属性是“学生选课”关系中的主属性，按照实体完整性约束，它们均不能取空值，所以“学生选课”关系中的“学号”和“课程号”属性实际上只能取相应被参照关系中已经存在的主码值。

在参照完整性约束中， R 与 S 可以是同一个关系。

例如对于例 2.3，按照参照完整性约束，“先修课”属性值可以取两类值：

- ① 空值，表示该门课程不存在先修课。
- ② 非空值，这时该值必须是本关系中某个元组的课程号。

2.3.3 用户定义的完整性

任何关系数据库系统都应该支持实体完整性和参照完整性，这是关系模型所要求的。除此之外，不同的关系数据库根据其应用场景不同，往往还需要一些特殊的约束条件。用户定义的完整性就是针对某一具体关系数据库的约束条件，它反映某一具体应用所涉及的数据必须满足的语义要求。例如，某个属性必须取唯一值，某个非主属性不能取空值等。例如，在例 2.1 的“学生”关系中，若要求学生必须有姓名，则可以定义“姓名”属性不能取空值；“学生选课”关系中“成绩”的取值范围可以定义为 0~100 等。

关系模型应提供定义和检验这类完整性约束的机制，以使用统一的系统的方法处理它们，而不需要由应用程序承担这一功能。

2.4 关系代数

关系代数是一种抽象的查询语言，它用对关系的运算来表达查询。

任何一种运算都是将一定的运算符作用于一定的运算对象上，得到预期的运算结果。所以运算对象、运算符、运算结果是运算的三大要素。

关系代数的运算对象是关系，运算结果亦为关系。关系代数用到的运算符包括两类：集合运算符和专门的关系运算符，如表 2.4 所示。

按运算符的不同，关系代数的运算可分为传统的集合运算和专门的关系运算两类。其中，传统的集合运算将关系看成元组的集合，其运算是从关系的“水平”方向，即行的角度来进行；专门的关系运算不仅涉及行，而且涉及列。比较运

表 2.4 关系代数运算符

运算符		含义
集合运算符	$\cup$	并
	$-$	差
	$\cap$	交
	$\times$	笛卡儿积
专门的关系运算符	σ	选择
	Π	投影
	$\bowtie$	连接
	$\div$	除

算符和逻辑运算符用于辅助专门的关系运算符进行操作。

2.4.1 传统的集合运算

传统的集合运算是二目运算,包括并、差、交、笛卡儿积4种运算。

设关系 R 和关系 S 具有相同的目 n (即两个关系都有 n 个属性),且相应的属性取自同一个域, t 是元组变量 ($t \in R$, 表示 t 是 R 的一个元组)。

可以定义并、差、交、笛卡儿积运算如下。

1. 并

关系 R 与关系 S 的并记作

$$R \cup S = \{t | t \in R \vee t \in S\}$$

其结果仍为 n 目关系,由属于 R 或属于 S 的元组组成。

2. 差

关系 R 与关系 S 的差记作

$$R - S = \{t | t \in R \wedge t \notin S\}$$

其结果关系仍为 n 目关系,由属于 R 而不属于 S 的所有元组组成。

3. 交

关系 R 与关系 S 的交记作

$$R \cap S = \{t | t \in R \wedge t \in S\}$$

其结果关系仍为 n 目关系,由既属于 R 又属于 S 的元组组成。关系的交可以用差来表示,即 $R \cap S = R - (R - S)$ 。

4. 笛卡儿积

这里的笛卡儿积严格地讲应该是广义笛卡儿积,因为这里笛卡儿积的元素是元组。

两个分别为 n 目和 m 目的关系 R 和 S , 其笛卡儿积是一个 $n+m$ 列的元组的集合。元组的前 n 列是关系 R 的一个元组,后 m 列是关系 S 的一个元组。若 R 有 k_1 个元组, S 有 k_2 个元组,则关系 R 和关系 S 的笛卡儿积有 $k_1 \times k_2$ 个元组。记作

$$R \times S = \{\widehat{t_i t_j} | t_i \in R \wedge t_j \in S\}$$

图 2.4 为关系 R 、 S 以及两者之间的传统集合运算示例。

2.4.2 专门的关系运算

专门的关系运算包括选择、投影、连接、除等运算。为了叙述上的方便,这里先引入几个记号。

① 设关系模式为 $R(A_1, A_2, \dots, A_n)$, 它的一个关系设为 R 。 $t \in R$, 表示 t 是 R 的一个元组。 $T[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量。

② 若 $A = \{A_{i1}, A_{i2}, \dots, A_{ik}\}$, 其中 $A_{i1}, A_{i2}, \dots, A_{ik}$ 是 $A_1, A_2, \dots, A_n$ 的一部分, 则称 A 为属性列或属性组。 $T[A] = (T[A_{i1}], T[A_{i2}], \dots, T[A_{ik}])$ 表示元组 t 在属性列 A 上诸分量的集合, $\bar{A}$ 则

R			S			$R \cup S$		
A	B	C	A	B	C	A	B	C
a_1	b_1	c_1	a_1	b_2	c_2	a_1	b_1	c_1
a_1	b_2	c_2	a_1	b_3	c_2	a_1	b_2	c_2
a_2	b_2	c_1	a_2	b_2	c_1	a_2	b_2	c_1
a_1	b_3	c_2				a_1	b_3	c_2

(a) (b) (c)

$R \cap S$			$R \times S$					
A	B	C	$R.A$	$R.B$	$R.C$	$S.A$	$S.B$	$S.C$
a_1	b_2	c_2	a_1	b_1	c_1	a_1	b_2	c_2
a_2	b_2	c_1	a_1	b_1	c_1	a_1	b_3	c_2
			a_1	b_1	c_1	a_2	b_2	c_1
			a_1	b_2	c_2	a_1	b_2	c_2
			a_1	b_2	c_2	a_1	b_3	c_2
			a_1	b_2	c_2	a_2	b_2	c_1
			a_2	b_2	c_1	a_1	b_2	c_2
			a_2	b_2	c_1	a_1	b_3	c_2
			a_2	b_2	c_1	a_2	b_2	c_1

(d) (e) (f)

$R - S$		
A	B	C
a_1	b_1	c_1

图 2.4 关系 R 、 S 以及两者之间的传统集合运算举例

表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i1}, A_{i2}, \dots, A_{ik}\}$ 后剩余的属性列。

③ R 为 n 目关系, S 为 m 目关系。 $t_r \in R$, $t_s \in S$, $\widehat{t_r t_s}$ 称为元组的连接(concatenation)或元组的串接。它是一个 $n+m$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组。

④ 给定一个关系 $R(X, Z)$, X 和 Z 为属性列。当 $t[X] = x$ 时, x 在 R 中的象集(images set)定义为

$$Z_x = \{t[Z] \mid t \in R, t[X] = x\}$$

它表示 R 中属性列 X 上值为 x 的诸元组在 Z 上分量的集合。

例如, 图 2.5 中,

$$x_1 \text{ 在 } R \text{ 中的象集 } Z_{x_1} = \{Z_1, Z_2, Z_3\}$$

$$x_2 \text{ 在 } R \text{ 中的象集 } Z_{x_2} = \{Z_2, Z_3\}$$

$$x_3 \text{ 在 } R \text{ 中的象集 } Z_{x_3} = \{Z_1, Z_3\}$$

R	
x_1	Z_1
x_1	Z_2
x_1	Z_3
x_2	Z_2
x_2	Z_3
x_3	Z_1
x_3	Z_3

图 2.5 象集举例

下面给出这些专门的关系运算的定义。

1. 选择(selection)

选择又称为限制(restriction)。它是在关系 R 中选择满足给定条件的诸元组, 记作

$$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{'真'}\}$$

其中 F 表示选择条件, 它是一个逻辑表达式, 取逻辑值“真”或“假”。

逻辑表达式 F 的基本形式为

$$X_1 \theta Y_1$$

其中 θ 表示比较运算符, 它可以是 $>$, $\geq$, $<$, $\leq$, $=$ 或 $<>$ 。 X_1 , Y_1 是属性名, 或为常量, 或为简单函数; 属性名也可以用其序号来代替。在基本的选择条件上可以进一步进行逻辑运算, 即进行求非($\neg$)、与($\wedge$)、或($\vee$)运算。条件表达式中的运算符如表 2.5 所示。

表 2.5 条件表达式中的运算符

运算符		含义
比较运算符	$>$	大于
	$\geq$	大于或等于
	$<$	小于
	$\leq$	小于或等于
	$=$	等于
	$<>$	不等于
逻辑运算符	$\neg$	非
	$\wedge$	与
	$\vee$	或

选择运算实际上是从关系 R 中选取使逻辑表达式 F 为真的元组。这是从行的角度进行的运算。

下面对 2.1.3 小节中图 2.1 的“学生选课”数据库中关系 Student、Course 和 SC 进行运算。

[例 2.4] 查询信息安全专业的全体学生。

$$\sigma_{\text{Smajor}=\text{'信息安全'}}(\text{Student})$$

结果如图 2.6(a) 所示。

[例 2.5] 查询 2001 年之后(包含 2001 年)出生的学生。

$$\sigma_{\text{Sbirthdate} \geq 2001-1-1}(\text{Student})$$

结果如图 2.6(b) 所示。

2. 投影(projection)

关系 R 上的投影是从 R 中选择若干属性列组成新的关系。记作

$$\pi_A(R) = \{t[A] | t \in R\}$$

其中 A 为 R 中的属性列。

投影操作是从列的角度进行的运算。

Sno	Sname	Ssex	Sbirthdate	Smajor
20180001	李勇	男	2000-3-8	信息安全

(a) 查询信息安全专业的全体学生

Sno	Sname	Ssex	Sbirthdate	Smajor
20180003	王敏	女	2001-8-1	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

(b) 查询2001年之后(包含2001年)出生的学生

图 2.6 选择运算示例

[例 2.6] 查询学生的学号和主修专业, 即求关系 Student 在“学号”和“主修专业”两个属性上的投影。

$$\Pi_{\text{Sno, Smajor}}(\text{Student})$$

结果如图 2.7(a) 所示。

投影不仅取消了原关系中的某些属性列, 还可能取消某些元组, 因为取消了某些属性列后就可能出现重复行, 应取消这些完全相同的行。

[例 2.7] 查询关系 Student 中的学生都主修了哪些专业, 即查询关系 Student 在“主修专业”属性上的投影。

$$\Pi_{\text{Smajor}}(\text{Student})$$

结果如图 2.7(b) 所示。关系 Student 原来有 7 个元组, 而投影结果取消了重复的元组, 因此最终结果只有 4 个元组。

学号 Sno	专业 Smajor
20180001	信息安全
20180002	计算机科学与技术
20180003	计算机科学与技术
20180004	计算机科学与技术
20180005	信息管理与信息系统
20180006	数据科学与大数据技术
20180007	数据科学与大数据技术

(a) 查询学生的学号和主修专业

专业 Smajor
信息安全
计算机科学与技术
信息管理与信息系统
数据科学与大数据技术

(b) 查询学生主修的专业

图 2.7 投影运算示例

3. 连接(join)

连接也称为 θ 连接, 指从两个关系的笛卡儿积中选取其属性间满足一定条件的元组。记作

$$R \bowtie_{A\theta B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}$$

其中, A 和 B 分别为关系 R 和 S 上列数相等且可比的属性列, θ 是比较运算符。具体来说, 连接运算是从笛卡儿积 $R \times S$ 中选取关系 R 在属性列 A 上的值与关系 S 在属性列 B 上的值满足比较关系 θ 的元组。

连接运算中有两种最为重要也最为常用的连接, 一种是等值连接(equijoin), 另一种是自然连接(natural join)。

θ 为“=”的连接运算称为等值连接。它是从关系 R 与 S 的广义笛卡儿积中选取 A 、 B 属性值相等的那些元组, 即

$$R \bowtie_{A=B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B] \}$$

自然连接是一种特殊的等值连接。它要求两个关系中进行比较的分量必须是同名的属性列, 并且在结果中把重复的属性列去掉。即若 R 和 S 中具有相同的属性列 B , U 为 R 和 S 的全体属性集合, 则自然连接可记作

$$R \bowtie S = \{ \widehat{t_r t_s} [U-B] \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B] \}$$

一般的连接操作是从行的角度进行运算, 但自然连接还需要取消重复属性列, 所以是同时从行和列的角度进行运算。

[例 2.8] 对于图 2.8(a)(b)所示的关系 R 和 S , 图 2.8(c)(d)(e)分别给出了非等值连接 $R \bowtie_{C < E} S$ 、等值连接 $R \bowtie_{R.B=S.B} S$ 和自然连接 $R \bowtie S$ 的结果。

R			S		$R \bowtie_{C < E} S$				
A	B	C	B	E	A	$R.B$	C	$S.B$	E
a_1	b_1	5	b_1	3	a_1	b_1	5	b_2	7
a_1	b_2	6	b_2	7	a_1	b_1	5	b_3	10
a_2	b_3	8	b_3	10	a_1	b_2	6	b_2	7
a_2	b_4	12	b_3	2	a_1	b_2	6	b_3	10
			b_5	2	a_2	b_3	8	b_3	10

(a) 关系 R

(b) 关系 S

(c) 非等值连接

$R \bowtie_{R.B=S.B} S$					$R \bowtie S$			
A	$R.B$	C	$S.B$	E	A	B	C	E
a_1	b_1	5	b_1	3	a_1	b_1	5	3
a_1	b_2	6	b_2	7	a_1	b_2	6	7
a_2	b_3	8	b_3	10	a_2	b_3	8	10
a_2	b_3	8	b_3	2	a_2	b_3	8	2

(d) 等值连接

(e) 自然连接

图 2.8 连接运算举例

两个关系 R 和 S 在做自然连接时, 选择两个关系在公共属性上值相等的元组构成新的关系。此时, 关系 R 中某些元组有可能在 S 中不存在公共属性上值相等的元组, 从而造成 R 中这些元组在操作时被舍弃了; 同样, S 中某些元组也可能被舍弃。这些被舍弃的元组称为悬浮元组(dangling tuple)。例如, 在图 2.8(e)的自然连接中, R 中的第 4 个元组、 S 中的第 5 个元组都是被舍弃的悬浮元组。

如果把悬浮元组也保存在结果关系中, 而在其他属性上填空值(NULL), 那么这种连接就叫作外连接(outer join), 记作 $R \bowtie S$; 如果只保留左边关系 R 中的悬浮元组就叫作左外连接(left outer join 或 left join), 记作 $R \ltimes S$; 如果只保留右边关系 S 中的悬浮元组就叫作右外连接(right outer join 或 right join), 记作 $R \rtimes S$ 。

例如, 图 2.9(a)是图 2.8 中关系 R 和关系 S 的外连接, 图 2.9(b)是左外连接, 图 2.9(c)是右外连接。

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

(a) 外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

(b) 左外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

(c) 右外连接

图 2.9 外连接运算举例

4. 除 (division)

设关系 R 除以关系 S 的结果为关系 T , 则 T 包含所有在 R 但不在 S 中的属性及其值, 且 T 的元组与 S 的元组的所有组合都在 R 中。

下面用象集来定义除法。

给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性列。 R 中的 Y 与 S 中的 Y 可以有不同的属性名, 但必须出自相同的域。

R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影: 元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合。记作

$$R \div S = \{t_r[X] \mid t_r \in R \wedge \Pi_Y(S) \subseteq Y_x\}$$

其中 Y_x 为 x 在 R 中的象集, $x = t_r[X]$ 。

除操作是同时从行和列角度进行运算。

[例 2.9] 设关系 R, S 分别为图 2.10 中的(a)和(b), $R \div S$ 的结果为图 2.10(c)。

在关系 R 中, A 可以取 4 个值 $\{a_1, a_2, a_3, a_4\}$ 。其中:

$$a_1 \text{ 的象集为 } \{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$$

a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$

a_3 的象集为 $\{(b_4, c_6)\}$

a_4 的象集为 $\{(b_6, c_6)\}$

S 在 (B, C) 上的投影为 $\{(b_1, c_2), (b_2, c_1), (b_2, c_3)\}$ 。

显然只有 a_1 的象集 $(B, C)_{a_1}$ 包含了 S 在 (B, C) 属性列上的投影, 所以

$$R \div S = \{a_1\}$$

R			S		
A	B	C	B	C	D
a_1	b_1	c_2	b_1	c_2	d_1
a_2	b_3	c_7	b_2	c_1	d_1
a_3	b_4	c_6	b_2	c_3	d_2
a_1	b_2	c_3			
a_4	b_6	c_6			
a_2	b_2	c_3			
a_1	b_2	c_1			

(a)

R÷S		
A		
a_1		

(c)

图 2.10 除运算举例

图 2.11 为例 2.9 的除运算过程示意。

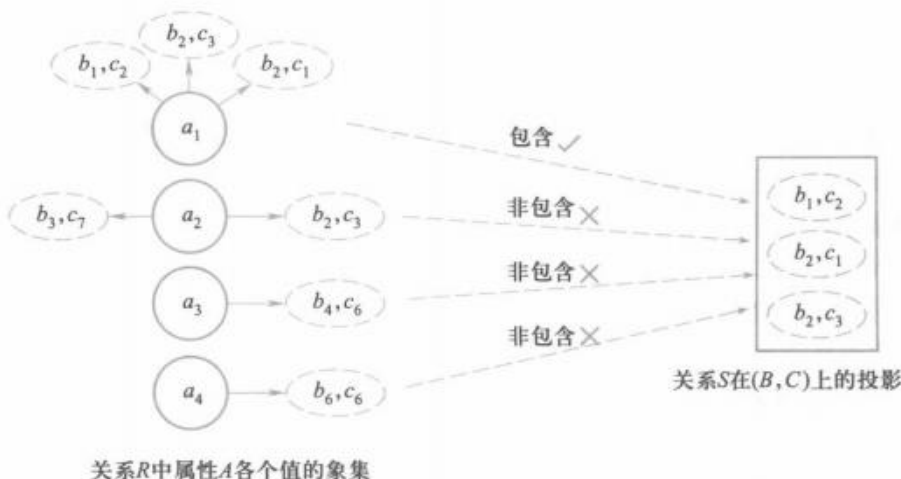


图 2.11 例 2.9 的除运算过程示意图

下面仍以 2.1.3 小节图 2.1 “学生选课”数据库为例, 给出几个综合应用多种关系代数运算进行查询的例子。

[例 2.10] 查询至少选修 81001 号课程和 81003 号课程的学生的学号。

首先建立一个临时关系 K :

K	
Cno	
81001	
81003	

然后求:

$$\Pi_{Sno, Cno}(SC) \div K = \{20180001, 20180002\}$$

求解过程与例 2.9 类似, 先对关系 SC 在 (Sno, Cno) 属性上投影, 然后逐一求出每一学生 (Sno) 的象集, 并依次检查这些象集是否包含 K 。

[例 2.11] 查询选修了 81002 号课程的学生的学号。

$$\Pi_{Sno}(\sigma_{Cno='81002'}(SC)) = \{20180001, 20180002, 20180003\}$$

[例 2.12] 查询至少选修了一门其直接先修课为 81003 号课程的学生的姓名。

$$\Pi_{Sname}(\sigma_{Cpno='81003'}(Course) \bowtie SC \bowtie \Pi_{Sno, Sname}(Student))$$

或

$$\Pi_{Sname}(\Pi_{Sno}(\sigma_{Cpno='81003'}(Course) \bowtie SC) \bowtie \Pi_{Sno, Sname}(Student))$$

[例 2.13] 查询选修了全部课程的学生的学号和姓名。

$$\Pi_{Sno, Cno}(SC) \div \Pi_{Cno}(Course) \bowtie \Pi_{Sno, Sname}(Student)$$

关系代数小结

本节介绍了 8 种关系代数运算, 其中并、差、笛卡儿积、选择和投影这 5 种运算为基本的运算。其他三种运算, 即交、连接和除, 均可以用这 5 种基本运算来表达。引进它们并不增加语言的运算能力, 但可以简化表达。

关系代数中, 这些运算经有限次复合而形成的表达式称为关系代数表达式。

此外, 还有扩展的关系代数, 如关系的重新命名、查询结果的去重操作、分组操作、排序操作和聚集函数等, 这里就不介绍了。

* 2.5 关系演算

关系演算是以数理逻辑中的谓词演算为基础的。按谓词变元不同, 关系演算可分为元组关系演算和域关系演算。本节通过元组关系演算语言 ALPHA 和域关系演算语言 QBE 来介绍关系演算的基本概念和查询方法。

* 2.5.1 元组关系演算语言 ALPHA

元组关系演算以元组变量作为谓词变元的基本对象。一种典型的元组关系演算语言是 Edgar F. Codd 提出的 ALPHA。这一语言虽然没有实际实现, 但关系数据库管理系统 INGRES 最初所用的 QUEL 语言就是参照 ALPHA 研制的, 与其十分类似。